LOAN MANAGEMENT SYSTEM

A MINI PROJECT REPORT

Submitted by

LINZA RS

23ALR046

ADHARSHINI KS

23ALR002

MONISH NARAIN N

23ALR053

in partial fulfilment of the requirements for

the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

NOVEMBER 2025

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

NOVEMBER 2025

BONAFIDE CERTIFICATE

This is to certify that the mini Project report entitled **LOAN MANAGEMENT SYSTEM** is by the bonafide record of the project work done by **ADHARSHINI K S (Register 23ALR002), LINZA R S (Register No:23ALR046), MONISH NARAIN (Register No:23ALR053)** in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in **Artificial Intelligence and Machine Learning** of Anna University Chennai during the year 2025-2026.

SUPERVISOR **HEAD OF THE DEPARTMENT**

Date: **(Signature with seal)**

Submitted for the end semester viva voce examination held on

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
KONGU ENGINEERING COLLEGE

(**Autonomous**)

PERUNDURAI ERODE – 638 060

NOVEMBER 2025

DECLARATION

We affirm that the Project Report titled **LOAN MANAGEMENT SYSTEM** being submitted in partial fulfillment of the requirements for the award of Bachelor of Technology is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion this or any other candidate.

Date:

LINZA R S
(23ALR046)

MONISH NARAIN N
(23ALR053)

ADHARSHINI K S
(23ALR02)

I certify that the declaration made by the above candidate is true to the best of my knowledge.

Date: Name and Signature of the Supervisor

ABSTRACT

The project "Loan Management System" focuses on creating a comprehensive digital platform that modernizes traditional loan processing and management. Manual loan handling often results in delays, errors, and limited accessibility for customers and administrators. To overcome these challenges, the system aims to fully digitize loan applications, approval workflows, customer record maintenance, and repayment tracking, ensuring greater efficiency, transparency, and convenience for all users involved.

The platform is built using modern technologies such as ReactJS for the frontend, providing an interactive and user-friendly interface, while Node.js manages backend operations with reliability and speed. MongoDB Atlas serves as a scalable and secure cloud database for storing customer profiles, loan information, and payment records. Additionally, the system integrates secure payment gateways to support smooth and reliable EMI transactions, ensuring a seamless financial experience for customers.

By offering features such as online loan applications, real-time approval status tracking, EMI scheduling, automated payment processing, and admin-side management tools, the Loan Management System simplifies complex workflows and reduces manual intervention. This system not only minimizes human errors but also enhances data security through automated verification and role-based access control. Overall, it provides a reliable, scalable, and future-ready solution capable of meeting the evolving needs of modern financial institutions and their customers.

ACKNOWLEDGEMENT

We express our sincere thanks and gratitude to **Thiru. E. R. K. Krishnan, M.Com.,** our beloved Correspondent, and all other philanthropic trust members of Kongu Vellalar Institute of Technology Trust who have always encouraged us in academic and co-curricular activities.

We are extremely thankful with no words of a formal nature to the dynamic Principal **Dr. R. Parameshwaran, M.E., Ph.D.,** for providing the necessary facilities to complete our work.

We would like to express our sincere gratitude to our respected Head of the Department **Dr. R. R. Rajalaxmi, M.E., PhD,** for providing the necessary facilities.

We extend our thanks to **Dr. K. Sathya, M.Tech.,Ph.D.,** Associate Professor Artificial Intelligence and Machine Learning, Project Coordinator for his encouragement and valuable advice that made us carry out the project work successfully.

We extend our gratitude to our Supervisor **Mrs. K . Senthilvadivu, B.E., M.E.,** Assistant Professor of Artificial Intelligence and Machine Learning, for his valuable ideas and suggestions, which have been very helpful in the project. We are grateful to all the faculty members of the Artificial Intelligence and Machine Learning Department for their support.

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	**iv**
	LIST OF FIGURES	**viii**
1.	**INTRODUCTION**	**1**
	1.1 Existing System	2
	1.2 System Study	2
	1.3 Objective	3
	1.4 Scope	3
2.	**GENERAL DESCRIPTION**	**4**
	2.1 Project Perspective	4
	2.2 User Characteristics	4
	2.3 Design And Implementation Constraints	5
3.	**REQUIREMENTS**	**6**
	3.1 Functional Requirements	6
	3.2 Non-Functional Requirements	6
	3.3 Software Components	7
	3.4 Hardware Components	8
4.	**DETAILED DESIGN**	**9**
	4.1 Architectural Design	9
	4.1.1 Module Characteristics	9
	4.1.1.1 User Module	9
	4.1.1.2 Admin Module	9

	4.1.1.2 Use Case Diagram	11
	4.1.1.3 Sequence Diagram	12
	4.2 Interface Design	13
	4.3 Database Design	13
	4.4 Output Design	14
5.	**MODULE DESCRIPTION**	**15**
	5.1 Module Characteristics	15
	5.1.1 Customer Module	15
	5.1.2 Loan Module	15
	5.1.3 GoldItem Module	15
	5.1.4 Payment Room	15
	5.1.5 Notification Module	16
	5.1.6 Admin Module	16
	5.1.7 AuditLogs Module	16
	5.1.8 Branch Module	16
	5.1.9 Loan Document Module	17
	5.1.10 Valuation Module	17
	5.1.11 EMI Module	17
	5.1.12 User Session Module	17
	5.1.13 Transaction Module	18
6.	**TESTING**	**19**
	6.1 Unit Testing	19
	6.2 Regression Testing	19

	6.3 Validation Testing	19
	6.4 Verification Testing	20
	6.5 Integration Testing	20
7.	**CONCLUSION AND FUTURE SCOPE**	**21**
	Appendix 1	22
	Appendix 2	44
	References	50

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Flow Chart	10
4.2	Use Case Diagram	11
4.3	Sequence Diagram	12
4.4	Database Design	14
5.1	Modules Diagram	19

CHAPTER 1

INTRODUCTION

"GoldFlow" is an advanced and intelligent digital platform specifically designed to modernize, simplify, and streamline the entire lifecycle of gold loan management for financial institutions as well as customers. The system functions as a secure, automated, and fully integrated solution that handles every stage of a gold-backed loan—from the initial application and valuation to approval, disbursement, repayment tracking, and final closure.

With its smart automation, GoldFlow allows banks, NBFCs, and financial agencies to efficiently manage customer records, securely store verification documents, and evaluate loan eligibility based on accurate gold purity testing and real-time valuation methods. The platform dynamically calculates interest rates, loan-to-value (LTV) ratios, processing fees, and EMI structures, ensuring that all financial computations remain precise and free from human error. It also generates clear and well-structured repayment schedules that borrowers can easily understand, helping them plan their finances better.

In addition to operational ease, GoldFlow promotes complete transparency by enabling smooth communication between lenders and borrowers through instant notifications, loan status updates, automated reminders, and digital receipts. Customers can conveniently apply for loans, track their application progress, check due dates, and make secure online payments—anytime and from anywhere. By replacing lengthy manual processes and paperwork with a fast, digital workflow, GoldFlow significantly reduces processing time, minimizes risks, and enhances overall accuracy.

It addresses this need by offering a robust, cloud-based platform that integrates seamlessly with existing banking systems while maintaining high standards of data security and regulatory compliance. Its automated processes not only reduce the workload on staff but also enable institutions to serve more customers in less time without compromising accuracy or service quality.

Furthermore, GoldFlow enhances decision-making and operational efficiency by providing advanced analytics, real-time dashboards, and automated reporting tools.

1.1 EXISTING SYSTEM

In many financial institutions, gold loan operations are still managed manually through ledgers or basic spreadsheets. This traditional approach requires staff to manually enter customer details, assess gold valuation, compute interest, and track repayment schedules. Such methods often result in data inconsistencies, delays in processing, and increased chances of human error. Additionally, the absence of centralized data makes it difficult to efficiently monitor active loans, pending renewals, and overdue accounts. Manual systems also fail to offer customer-friendly features like real-time loan tracking, instant updates, or automated notifications. By centralizing data and automating all major calculations and workflows, GoldFlow ensures consistent loan processing, reduces operational workload, and enhances accountability at each stage of the loan cycle.

1.2 SYSTEM STUDY

1.2.1 Understanding the Business Requirements

The primary requirement of the Gold Loan Management System is to enable financial organizations to process and manage gold loans digitally. The system must handle customer registration, gold valuation, loan approval, repayment tracking, and notifications efficiently while maintaining security and regulatory compliance.

1.2.2 Analyzing the Existing Systems and Processes

The existing manual process lacks automation in gold appraisal and interest calculation. Loan officers must evaluate gold manually, verify customer eligibility, and update records by hand, which is time-consuming and error-prone. A detailed analysis of these workflows helped in designing a digital system that automates core operations while maintaining transparency and control.

1.2.3 Identifying User Needs

Users of the system include customers, employees, and administrators. Customers need an easy-to-use interface to apply for loans, track payments, and request renewals. Bank employees need modules to approve or close loans, verify gold valuation, and manage repayments. Administrators require analytical dashboards to monitor loan performance.

1.2.4 Developing Use Cases

Use cases were developed to illustrate the main workflows—loan creation, gold valuation, payment recording, and loan renewal. These diagrams define how each user interacts with the system, helping developers design efficient modules and preventing redundancy or workflow conflicts.

1.3 OBJECTIVE

The primary objective of the GoldFlow – Gold Loan Management System is to create a comprehensive digital platform that automates and secures every stage of the gold loan process for institutions and customers.It replaces traditional manual procedures with an integrated solution that enhances accuracy, transparency, and efficiency. GoldFlow streamlines the entire gold loan lifecycle—from customer registration and gold valuation to approval, repayment tracking, renewal, and closure.It offers a centralized environment where all loan-related data, calculations, and documents are managed systematically to reduce errors and eliminate paperwork.The system automates gold valuation and interest calculations using real-time market rates for accurate and compliant loan processing..

1.4 SCOPE

The scope of the GoldFlow system covers the complete lifecycle of gold loans—from application and valuation to repayment and closure. It caters to customers applying for loans, branch employees managing transactions, and administrators monitoring the entire system. The platform calculates gold valuation based on purity, weight, and market rate, determines the eligible loan amount, and automatically computes interest, EMI, and penalties. It also generates reminders for due payments, renewal alerts, and overdue notifications. GoldFlow ensures scalability, allowing integration with multiple branches and customer accounts. It supports secure authentication, role-based access control, and real-time database operations. In addition, the system provides analytical insights such as total loans disbursed, repayment trends, and outstanding amounts. With its responsive web interface and automated backend logic, the system promotes transparency, efficiency, and trust between lenders and borrowers, offering a complete solution for modern gold loan management.

r.

CHAPTER 2

GENERAL DESCRIPTION

2.1 PROJECT PERSPECTIVE

- Manual or semi-digital gold loan processes often lead to delays, errors, and poor tracking for financial institutions. GoldFlow overcomes these challenges by bringing all essential operations into a single digital workflow.
- React powers the frontend, providing a smooth and responsive user interface for all loan-related operations. Node.js and Express manage the backend logic securely and efficiently, while MongoDB ensures reliable data storage.
- The platform is designed to scale efficiently, allowing financial institutions to manage multiple branches and increasing loan volumes without performance issues. It also ensures strict compliance with essential banking standards such as KYC verification.

2.2 USER CHARACTERISTICS

For The success of the GoldFlow system largely depends on its ability to meet the needs of its primary users: customers, loan officers, and administrators. Each group interacts with the platform in distinct ways based on their responsibilities and expectations. Customers are individuals seeking gold-backed loans who require a secure and convenient digital process. They should be able to submit personal and gold-related details online without relying on lengthy paperwork. Customers can apply for loans, receive quick assessments, and view gold valuation reports directly through the platform. They can also track approved loan amounts, monitor repayment schedules, and make repayments easily. The system allows them to request loan renewals and stay updated through a simple, user-friendly interface. Automated notifications alert customers about due dates, interest updates, and renewal opportunities. These features ensure transparency and help build trust between the institution and the borrower. Loan officers depend on GoldFlow to evaluate loan applications accurately and efficiently. They can automatically calculate gold values, EMIs, interest rates, and penalties using real-time data. A centralized dashboard helps officers track pending, approved, and closed loans without confusion. They can instantly update repayment records, renewal details, and maintain complete transaction histories.

2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS

2.3.1 Time

The project must adhere to strict timelines for design and deployment to ensure the timely launch of the gold loan management system. Any delay in meeting project milestones could impact the integration of key modules such as loan applications.

2.3.2 Budget Limitations

Budget constraints may limit the availability of resources for development, hosting, licensing of APIs, or the integration of advanced features such as real-time gold market updates or secure payment gateways.

2.3.3 Security

It handles sensitive data, including personal customer information, loan details and gold valuations, the system must comply with strict security standards. Implementation constraints may arise due to regulatory compliance requirements.

2.3.4 Scalability

The architecture of GoldFlow must be scalable to accommodate future growth in users, loan transactions and institutional branches. Scalability constraints could emerge if the system is not designed to handle an increasing number of concurrent users, higher data volumes, or more complex reporting and analytics requirements.

CHAPTER 3

REQUIREMENTS

3.1 FUNCTIONAL REQUIREMENTS

3.1.1 User Authentication

The GoldFlow system ensures secure user authentication through encrypted passwords, safe login/logout processes. Users receive role-based access where customers manage loan activities, while loan officers and administrators handle approvals and records.

3.1.2 Loan Application and Gold Valuation

Application allows customers to apply for gold loans online by submitting personal details, gold specifications. The system automatically appraises the gold using current market rates, calculates eligible loan amounts, provides real-time application status updates.

3.1.3 Repayment and Renewal Management

The system offers an interactive interface for customers to view repayment schedules and request loan renewals, with automatic calculations. Lenders can monitor active and overdue loans in real time, update records, and send automated reminders.

3.1.4 Reporting and Analytics

Administrators can access comprehensive reporting and analytics tools that offer detailed insights into loan trends, customer repayment performance and overdue accounts. Interactive dashboards display visual summaries and key performance indicators, helping administrators quickly assess operations

3.1.5 Notifications and Alerts

GoldFlow includes an automated notification system that alerts customers about loan approvals, repayment due dates, interest updates. Loan officers and administrators receive notifications for pending approvals, overdue loans, and system updates.

3.2 NON-FUNCTIONAL REQUIREMENTS

3.2.1 Performance

The GoldFlow system should load quickly and respond promptly to user interactions, even when handling multiple concurrent users. Backend processing for loan calculations, gold valuations, and report generation should be optimised to minimise latency.

3.2.2 Usability

The platform should be intuitive and user-friendly, with clear navigation and role-specific dashboards for customers, loan officers, and administrators. User interfaces should display loan details, gold valuations, repayment schedules.

3.2.3 Reliability

The system should be stable and reliable, providing consistent performance even during unexpected spikes in usage. Robust error handling mechanisms must be in place to prevent data loss, corruption, or failed transactions.

3.2.4 Availability

GoldFlow should maintain high availability with minimal downtime to ensure uninterrupted service for all stakeholders. Redundant infrastructure and automated data backups should be in place to maintain system accessibility even during software failures.

3.3 SOFTWARE COMPONENTS

The software components of this project include a user-friendly front-end interface built using ReactJS, a secure and scalable back-end developed with Node.js and Express, and a cloud-based MongoDB Atlas database for storing customer and loan information.

3.3.1 Frontend

The frontend of the system is developed using ReactJS, providing a fast, interactive and user-friendly interface for both customers and administrators.

3.3.2 Backend:

The backend is built using Node.js and Express, offering a secure, scalable, and high-performance server environment. It handles business logic, processes loan applications, performs interest and LTV calculations, manages authentication, and ensures secure communication between the frontend and the database.

- **Database:**

  The system uses MongoDB Atlas, a cloud-based NoSQL database, to store and manage customer information, loan records, gold valuation data, and transaction logs.

- **DeploymentTools:**

  GoldFlow uses modern deployment tools to ensure smooth delivery and scalability. Docker is used for containerizing the application, allowing it to run consistently across different environments. Kubernetes manages container orchestration and scaling.

- **Monitoring & Maintenance Tools:**

  GoldFlow uses powerful monitoring and maintenance tools to ensure continuous performance and reliability.

3.4 HARDWARE COMPONENTS

Hardware components are the essential physical resources required to develop, deploy, and operate the GoldFlow digital gold loan management system**.**

- **Processor:** Intel Core i5 / i7 or AMD Ryzen 5 and above
- **RAM:** Minimum 8 GB (16 GB recommended for smooth performance)
- **Storage:** 256 GB SSD or higher for fast data access
- **Network:** Stable high-speed internet connection (minimum 10 Mbps) for cloud interaction.
- **Operating System:** Windows 10 / 11, Ubuntu 20.04+, or macOS
- **Server:** Render.com cloud server (512 MB – 2 GB RAM) with MongoDB Atlas and Redis support
- **Peripherals:** Keyboard, mouse, and monitor (1080p resolution)

CHAPTER 4

DETAILED DESIGN

4.1 ARCHITECTURAL DESIGN

The architectural design of the GoldFlow System illustrates how various components of the gold loan management process interact with each other. It defines the structure, modules, interfaces, and data flow between the client-side and server-side components. The system architecture includes several UML diagrams such as use case diagrams, sequence diagrams, and activity diagrams for both user and admin modules.

4.1.1 MODULE CHARACTERISTICS

The system contains two primary modules:

- User module
- Admin module

4.1.1 User Module

The user module serves as the primary interaction layer for borrowers, giving them smooth and convenient access to the loan system's key services. This module consists of several essential pages, including the Home page, Apply for Loan page, View Loan Status page, About Us page, and Contact Us page. Each page is designed with a clear layout and intuitive interface to ensure that users can easily understand and navigate the platform without confusion. Through the Home page, borrowers get an overview of available features, while the Apply for Loan page guides them step-by-step in submitting their loan applications. The View Loan Status page allows customers to track the progress of their applications and stay updated on approvals or pending actions, Figure 4.1

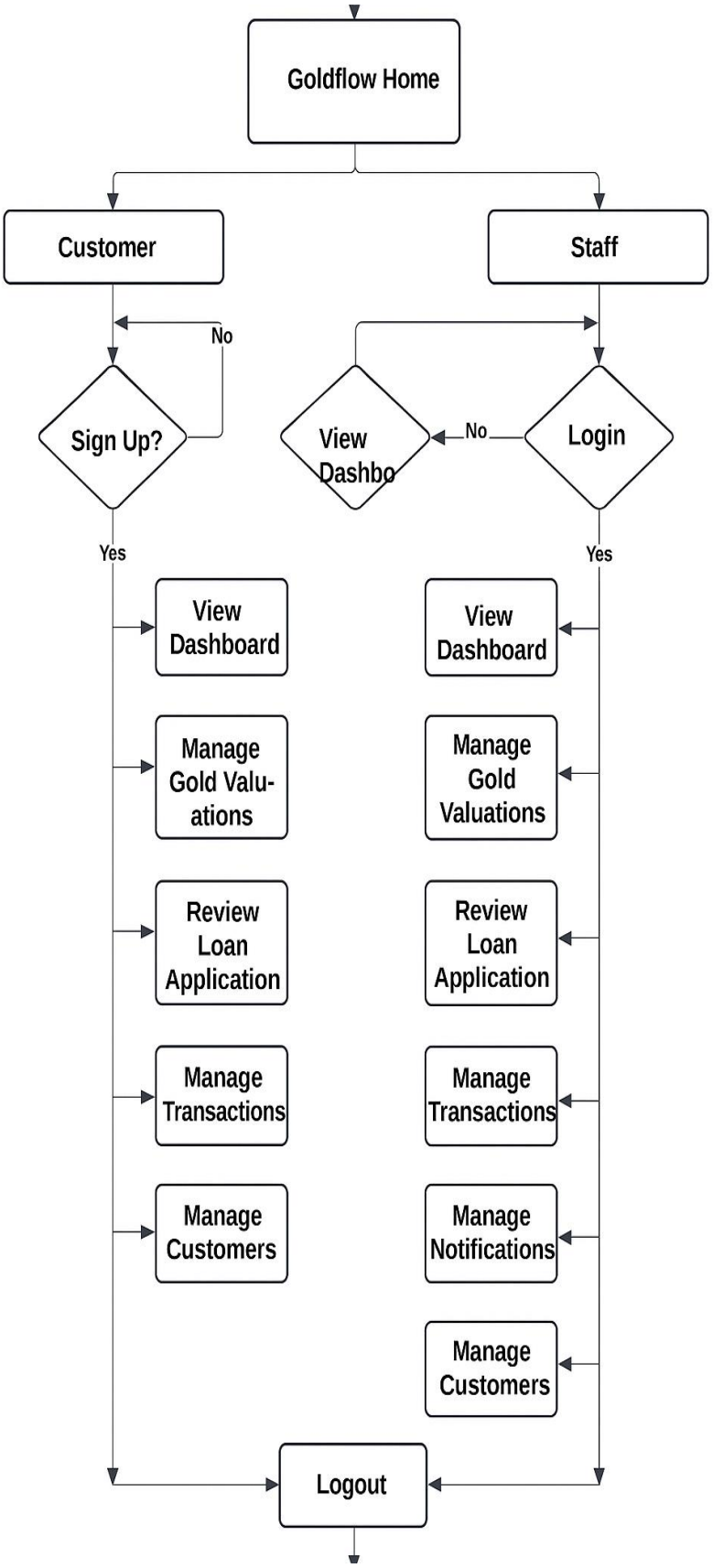

Figure 4.1 Flow Chart

4.1.1.2 USE CASE DIAGRAM

The Admin Module serves as the central control unit of the loan management system, providing complete oversight and administrative authority to ensure smooth and secure operations. Through this module, authorized administrators can manage critical tasks such as verifying customer information, performing gold valuation, approving or rejecting loan applications, and overseeing daily transactions. It includes several important functionalities, such as the Admin Login Page that ensures secure access, and the Manage Customers section where administrators can add, update, or remove customer profiles as needed. The Gold Valuation Management feature allows admins to input or modify gold rates and valuation details to maintain accurate lending calculations. Administrators can also review and approve loan applications by validating customer data and system-generated valuations. Figure 4.2

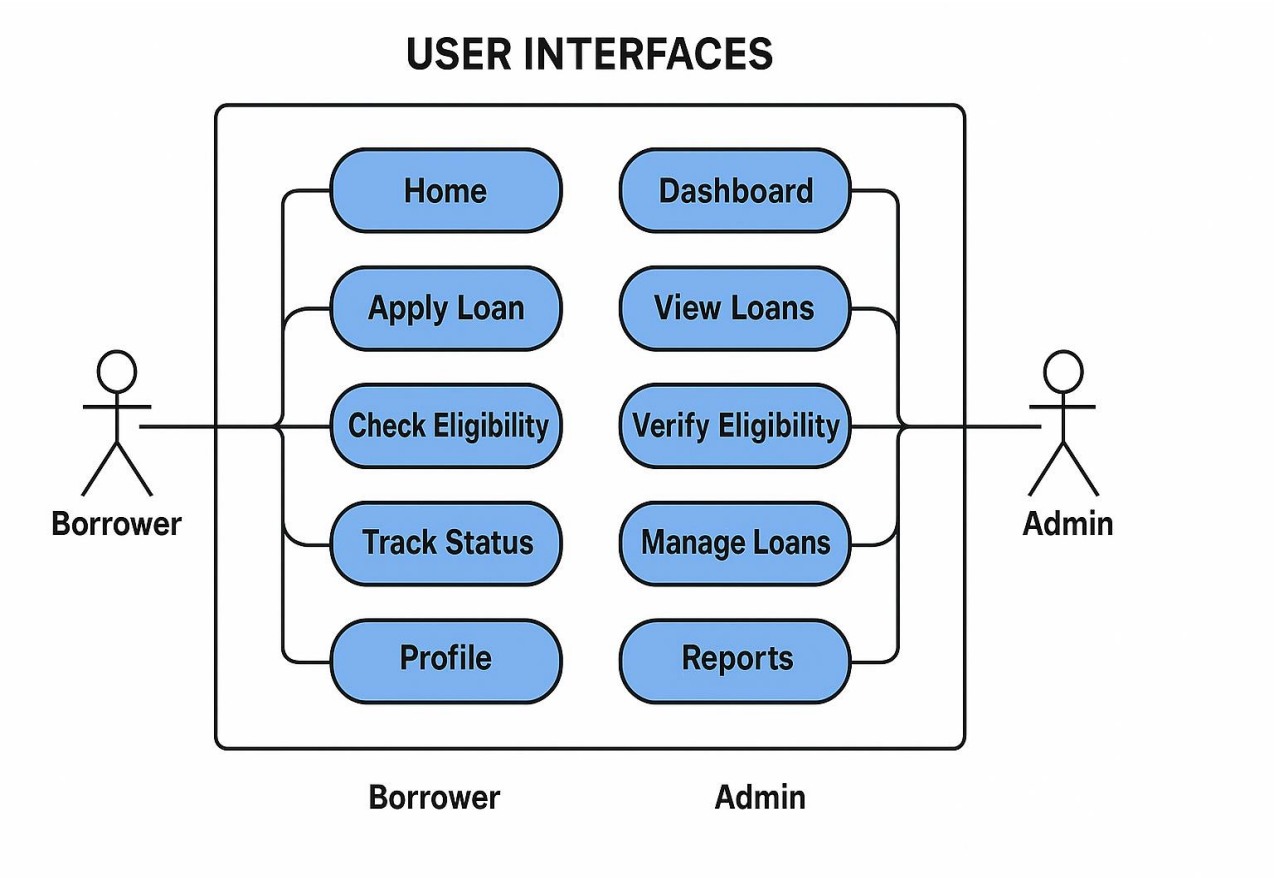

Figure 4.2 Use Case Diagram

4.1.1.3 SEQUENCE DIAGRAM

A sequence chart diagram is one of the five UML diagrams used to model the dynamic nature of a system. In the sequence diagram for the Loan Management System, the customer initiates actions such as login, applying for a loan, and checking loan status. The system validates the input, stores the request, and forwards it to the admin for verification and approval. Once approved, the system updates the loan status and generates an EMI schedule. Admins can view loan applications, approve or reject them, and manage repayment updates. Figure 4.3

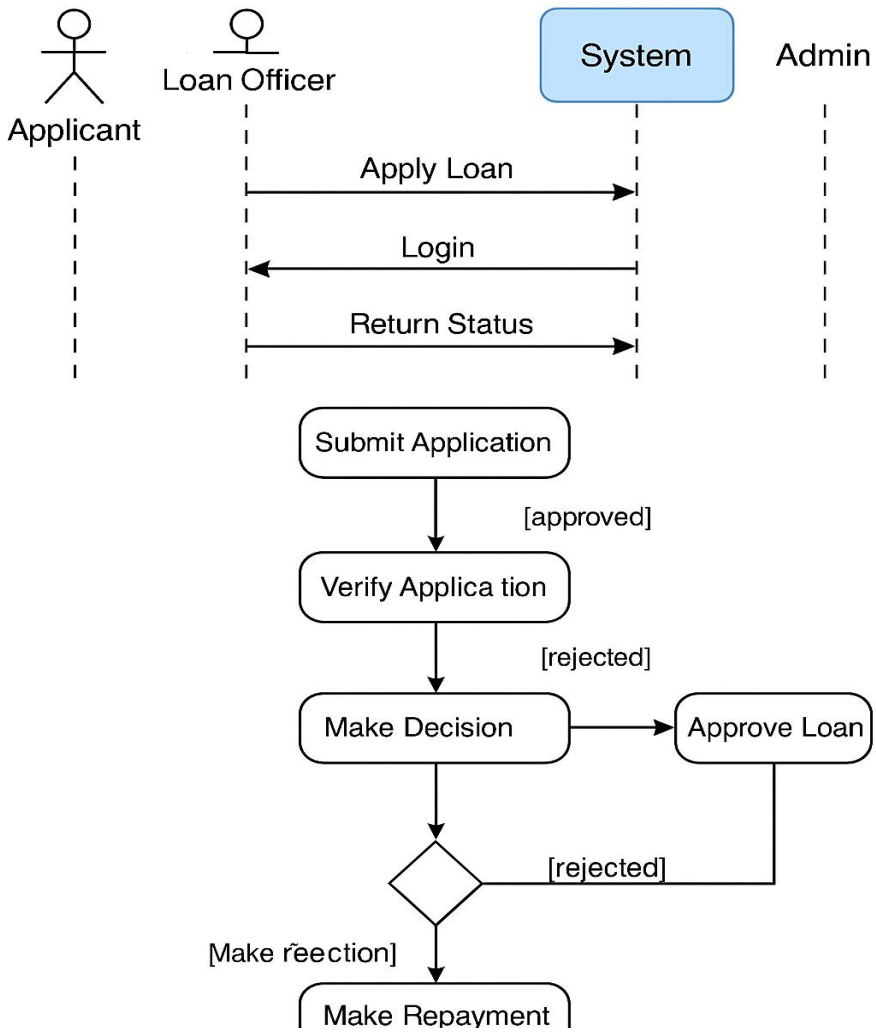

Figure 4.3 Sequence Diagram

4.2 INTERFACE DESIGN

The Loan Management System provides interactive and user-friendly interfaces for both Customer and Admin modules, focusing on clarity, usability, and responsiveness across desktops, tablets, and mobile devices. The Customer Interface allows users to navigate essential features such as Home, Login/Registration, Loan Application, Loan Status tracking, Payment Schedules, and Feedback submission, all organized in a simple and intuitive layout for smooth interaction. Consistent design elements like structured menus, clear buttons, and responsive forms ensure quick access and a visually appealing experience. On the administrative side, the Admin Interface offers secure access for authorized personnel to view customer applications, approve or reject loan requests, track loan disbursements, monitor repayments, and manage customer records efficiently. It also provides tools for handling feedback, generating analytical reports, and updating loan terms based on institutional policies. With its dashboard-style layout, graphical summaries, and neatly labelled panels, the admin environment supports fast decision-making and efficient oversight. Both interfaces are designed with ease of use, consistency, accessibility, and responsiveness in mind, ensuring that users with varying technical skills can operate the system comfortably..

4.3 DATABASE DESIGN

The Database The Loan Management System uses MongoDB as its database, a NoSQL platform that stores information in flexible document-based collections. The system includes several key collections such as Customers, which holds personal and account details; Admins, which stores secure login credentials and role data; Loan_Applications, which contains information about loan requests including type, amount, tenure, and approval status; and Loan_Payments, which records EMI transactions, payment dates, and outstanding balances. Additional collections like Feedbacks maintain customer reviews and ratings, while Notifications manage system alerts and communication between customers and administrators. MongoDB provides scalability, flexibility, and reliable real-time data handling, ensuring smooth performance and efficient information retrieval across all modules of the system. Figure 4.4

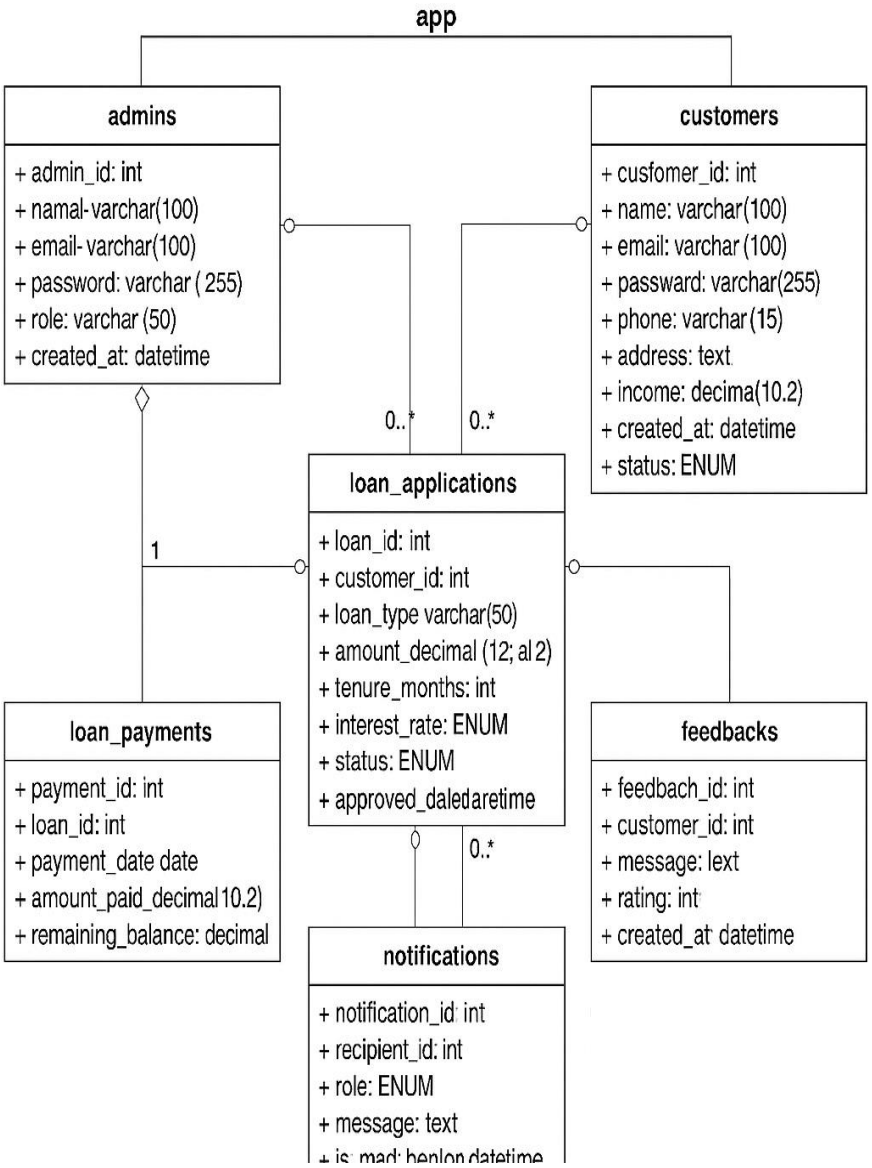

Figure 4.4 Database Diagram

4.4 OUTPUT DESIGN

The output design focuses on providing clear, informative, and well-structured results for end-users. Outputs are generated in the form of loan approval status, EMI schedules, payment summaries, and reports for admins. For customers, the system generates outputs such as loan approval notifications, payment reminders, and account summaries. For admins, the outputs include dashboards with total applications, approved loans, pending reviews, and revenue reports.

CHAPTER 5

MODULE DESCRIPTION

5.1 MODULE CHARACTERISTICS

5.1.1 Customer Module

The Customer model represents individuals who apply for or hold gold loans in the GoldFlow system. It includes fields such as name, mobile number, email, address, KYC details, and account credentials. It is used to securely store and manage customer identities, enabling user authentication, loan application tracking, and personalized access to loan information.

5.1.2 Loan Module

The Loan model represents a gold-backed loan created for a customer. It contains fields such as gold weight, purity, valuation amount, sanctioned amount, interest rate, LTV ratio, disbursement date, EMI schedule, and loan status. It is used to store and process all loan-related information, helping the system calculate repayments, generate schedules, monitor active loans, and manage renewals or closures.

5.1.3 GoldItem Module

The GoldItem model stores details of the gold ornaments or assets submitted as collateral. It includes fields such as item type, weight, purity (karat), valuation report, photographs, and appraisal date. It ensures accurate tracking and verification of the collateral, supports valuation-based loan calculations, and maintains transparency in the gold assessment process.

5.1.4 Payment Module

The Payment model represents transactions made by customers toward loan repayment. It typically includes the amount paid, payment method, date/time, reference number, and the loan ID it belongs to. It is used to track repayment history, update outstanding balances, generate receipts, and ensure proper financial reconciliation within the system.

5.1.5 Notification Module

The Notification model manages system alerts sent to customers and administrators. It contains fields such as notification type, message content, target user, timestamp, and related loan or payment references. It is used to deliver updates such as EMI reminders, loan approval alerts, overdue warnings, and system messages, supporting real-time communication and customer engagement**.**

5.1.6 Admin Module

The Admin model represents authorized staff or financial agents managing the loan system. It includes login credentials, role/permissions, branch detail. It allows administrators to review loan applications, validate gold valuations, approve loans, manage customer accounts, and oversee overall system operations with secure role-based access. Enables system configuration management, such as updating gold rate settings, interest configurations, and notification templates. Provides access to dashboard analytics, showing loan trends, customer growth, branch performance, and repayment status.

5.1.7 AuditLog Module

The AuditLog model records system activities for tracking, accountability, and compliance. It stores fields such as action type, user (customer/admin), timestamp, and description of the event. It helps maintain transparency and ensures compliance by logging actions like loan approvals, edits, payments, and account changes for future verification. Logs system errors, failed login attempts, and unauthorized operations for enhanced security insights. Supports exporting log data for auditing, compliance reporting, and regulatory verification.

5.1.8 Branch Module

The Branch model stores information about the physical or operational branches managing gold loan services. It includes fields such as branch name, location, branch code, contact details, operating hours, and assigned staff. This model helps organize operations across multiple branches, supports loan routing, and enables administrators to track branch-specific performance, loan distributions, and workflow activities**.** Tracks branch performance metrics such as total loans issued, repayments collected, and NPA (Non-Performing Asset) status. Supports branch-wise staff mapping, helping manage which employees are responsible for valuation, approvals.

5.1.9 Loan Document Module

The LoanDocument model manages all documents submitted and generated for each loan application. It stores information such as document type (ID proof, address proof, gold valuation certificate), document file or URL, upload timestamp, and associated loan ID. This model ensures secure storage of sensitive customer and loan documents, provides easy retrieval for verification, and maintains an audit trail for regulatory compliance. It supports document validation workflows, tracks approvals, and integrates with the Loan.

5.1.10 Valuation Module

The Valuation model manages gold valuation records used to calculate loan eligibility. It contains fields such as gold weight, purity test results, market rate at the time of valuation, valuation officer details, and valuation report. It ensures accurate assessment of gold assets, supports fair loan calculations, and provides an audit trail for valuation-related decisions to maintain trust and transparency. Helps auditors verify whether the loan amount aligns with real-time market rates and purity tests. Supports revaluation cycles, particularly for renewed or extended loans

5.1.11 EMI Module

The EMI model represents detailed Equated Monthly Installment schedules generated for each loan. It includes fields such as installment number, due date, principal amount, interest amount, total EMI amount, and payment status. It helps customers track repayment cycles, assists the system in calculating outstanding balances, and allows administrators to monitor overdue EMIs for timely follow-up and notifications. Supports notification workflows for upcoming due dates and overdue EMIs. Ensures financial transparency by offering customers a detailed breakdown of each EMI component.

5.1.12 User Session Module

The UserSession model stores data related to user logins and active sessions within the GoldFlow system. It includes session ID, user ID (customer/admin), login timestamp, logout timestamp, IP address, and device details. This model enhances security by monitoring user activity, detecting unauthorized access, maintaining session logs, and supporting features such as auto-logout or multi-device login restrictions.

5.1.13 Transaction Module

The Transaction model records all financial activities occurring within the GoldFlow system beyond regular EMI payments. It includes fields such as transaction type (disbursement, part-payment, penalty, refund), amount, date/time, reference ID, payment channel, and associated loan or customer ID. This model ensures accurate financial tracking, supports reconciliation, provides transparency for audits, and helps administrators review the complete monetary flow within the system.Figure 6.1

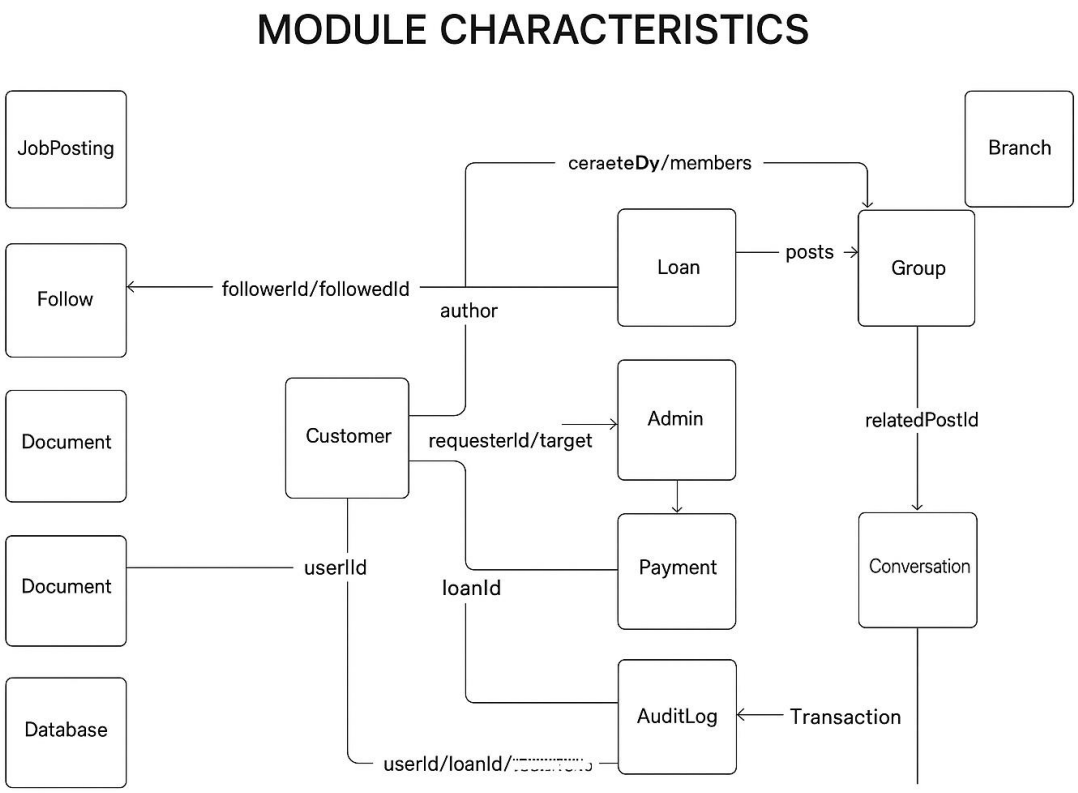

Figure 5.1 Modules Diagram

CHAPTER 6

TESTING

6.1 UNIT TESTING

Unit testing is a fundamental software testing process in which individual and smallest testable components of the application, called units or modules, are tested independently to ensure they function as intended. In the Loan Management System, each module—such as Customer Registration, Loan Application, Payment Schedule, Loan Status, and Admin Approval—is tested individually to verify its correctness and reliability.

6.2 REGRESSION TESTING

Regression testing is performed to ensure that recent changes, enhancements, or bug fixes in the system do not introduce new errors or affect existing functionality. In the Loan Management System, regression testing involves re-executing previously performed test cases on modules such as Loan Status, EMI Payment, Admin Approval, and Reports. The main goal of regression testing is to confirm that the system continues to function correctly and that all previously implemented features—like loan application submission, payment history updates, and customer feedback processing—remain unaffected by modifications.

6.3 VALIDATION TESTING

Validation testing ensures that the Loan Management System (LMS) meets all defined business requirements and fulfills user expectations by verifying whether the developed system performs accurately, reliably, and as intended under real-world conditions, essentially answering the question, “Are we building the right system?” It confirms that the final software aligns with the specifications defined during the analysis and design phase by evaluating both functional and non-functional aspects through realistic input and user interactions.

6.4 VERIFICATION TESTING

Verification testing is the process of evaluating each module during its development stage to ensure that it meets all specified design and technical requirements, essentially answering the question, “Are we building the system correctly?” In the Loan Management System, this includes checking that input validations in both the Customer and Admin modules function properly, ensuring the accuracy of calculation logic for loan amounts, interest, and EMIs, and confirming that user interfaces remain consistent with fully functional navigation links. It also involves validating that all database operations—such as storing loan applications, updating payment histories, and retrieving records—work correctly without errors. Through this process, verification ensures that every module is implemented exactly according to the design specifications before being integrated into the complete system.

6.5 INTEGRATION TESTING

Integration testing focuses on evaluating how multiple modules interact to ensure they work together as a complete and unified system. In the Loan Management System, once individual modules are verified, integration testing is conducted to validate combined workflows such as a customer submitting a loan application, the admin reviewing and approving or rejecting it, and the updated status reflecting accurately on the customer portal. It also examines the generation of payment schedules, customer EMI payments, admin tracking of repayment history, and automatic updates in system reports. This level of testing helps identify interface issues, data flow inconsistencies, and logic conflicts between connected modules. Overall, integration testing ensures smooth communication across components and confirms that the system delivers a seamless experience for both customers and administrators..

CHAPTER 7

CONCLUSION AND FUTURE WORK

The development of the Loan Management System (LMS) marks an important advancement in modernizing financial services and improving accessibility for customers and administrators. By integrating technologies such as ReactJS for a responsive user interface, Node.js for scalable backend processing, MongoDB for secure and flexible data storage, and online payment gateways for EMI handling, the system streamlines loan-related activities. This technology-driven approach enhances operational efficiency and delivers a smooth, fully digital experience for all users.

The core purpose of the LMS is to simplify loan applications, ensure timely approvals, and maintain complete transparency in tracking loan payments. Customers benefit from an intuitive platform where they can submit loan requests, check approval status, view detailed repayment schedules, and provide feedback. At the same time, administrators can manage loan applications, monitor repayments, generate performance reports, and maintain customer records securely, allowing for improved accuracy, reduced manual effort, and better decision-making.

To remain effective in a competitive financial landscape, the LMS will continue to evolve through future enhancements and technological upgrades. Planned improvements include automated risk assessment using AI to analyze loan eligibility and detect fraud, advanced analytics dashboards for insights on loan trends and repayment behavior, and development of dedicated mobile applications to provide anytime, anywhere access for customers. With these enhancements, the Loan Management System is positioned to transform traditional lending into a fully digital, efficient, and customer-centric process.

.

APPENDIX 1

CODING:

User.js:

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const userSchema = new mongoose.Schema({
name: {
type: String,
required: [true, 'Name is required'],
trim: true,
maxlength: [100, 'Name cannot exceed 100 characters']
},
email: {
type: String,
required: [true, 'Email is required'],
unique: true,
lowercase: true,
match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/, 'Please enter a valid email']
},
phone: {
type: String,
required: [true, 'Phone number is required'],
unique: true,
match: [/^[6-9]\d{9}$/, 'Please enter a valid 10-digit phone number']
},
password: {
type: String,
required: [true, 'Password is required'],
minlength: [6, 'Password must be at least 6 characters'],
select: false
},
```

```
role: {
type: String,
enum: ['ADMIN', 'USER', 'PENDING'],
default: 'PENDING'
},
address: {
type: String,
trim: true,
maxlength: [200, 'Address cannot exceed 200 characters']
},
city: {
type: String,
trim: true,
maxlength: [50, 'City cannot exceed 50 characters']
},
pincode: {
type: String,
trim: true,
match: [/^\d{6}$/, 'Please enter a valid 6-digit pincode']
},
aadharNumber: {
type: String,
trim: true,
match: [/^\d{12}$/, 'Please enter a valid 12-digit Aadhar number'],
sparse: true
},
panNumber: {
type: String,
trim: true,
uppercase: true,
match: [/^[A-Z]{5}[0-9]{4}[A-Z]{1}$/, 'Please enter a valid PAN number'],
sparse: true
},
```

```
isActive: {
type: Boolean,
default: true
},
approvedAt: {
type: Date
},
approvedBy: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User'
},
rejectionReason: {
type: String,
trim: true
}
}, {
timestamps: true
});
// Hash password before saving
userSchema.pre('save', async function(next) {
if (!this.isModified('password')) return next();
try {
const salt = await bcrypt.genSalt(12);
this.password = await bcrypt.hash(this.password, salt);
next();
} catch (error) {
next(error);
}
});
// Compare password method
userSchema.methods.comparePassword = async function(candidatePassword) {
return await bcrypt.compare(candidatePassword, this.password);
};
```

```
// Virtual field to map _id to id
userSchema.virtual('id').get(function() {
return this._id.toHexString();
});
// Ensure virtual fields are serialized
userSchema.set('toJSON', {
virtuals: true
});
// Remove password from JSON output
userSchema.methods.toJSON = function() {
const user = this.toObject();
delete user.password;
return user;
};
module.exports = mongoose.model('User', userSchema);
```

Admin.js:

```
const express = require('express');
const {
getDashboardData,
getAllCustomers,
getAllLoansWithFilters,
getSystemSettings,
updateSystemSettings,
getAllRenewalRequests,
updateRenewalStatus,
getSystemStatistics
} = require('../controllers/adminController');
const { protect, authorize } = require('../middleware/auth');
const router = express.Router();
// All routes are protected and admin only
router.use(protect);
router.use(authorize('ADMIN'));
// Admin routes
```

```
router.get('/dashboard', getDashboardData);
router.get('/customers', getAllCustomers);
router.get('/loans', getAllLoansWithFilters);
router.get('/settings', getSystemSettings);
router.put('/settings', updateSystemSettings);
router.get('/renewals', getAllRenewalRequests);
router.put('/renewals/:id/status', updateRenewalStatus);
router.get('/statistics', getSystemStatistics);
module.exports = router;
```

Auth.js:

```
const express = require('express');
const {
register,
login,
getMe,
logout,
getPendingUsers,
approveUser
} =
require('../controllers/authController');
const { protect, authorize } = require('../middleware/auth');
const { validateUserRegistration, validateUserLogin } = require('../middleware/validation');
const router = express.Router();
// Public routes
router.post('/register', validateUserRegistration, register);
router.post('/login', validateUserLogin, login);
// Protected routes
router.get('/me', protect, getMe);
router.post('/logout', protect, logout);
// Admin routes
router.get('/pending-users', protect, authorize('ADMIN'), getPendingUsers);
router.put('/approve-user/:id', protect, authorize('ADMIN'), approveUser);
module.exports = router;
```

Loans.js:

```
const express = require('express');
const {
getAllLoans,
getUserLoans,
getLoan,
createLoan,
updateLoan,
deleteLoan,
updateLoanStatus,
checkOverdueLoans
} = require('../controllers/loanController');
const { protect, authorize, requireApproval} = require('../middleware/auth');
const { validateLoanCreation } = require('../middleware/validation');
const router = express.Router();
// All routes are protected and require approval
router.use(protect);
router.use(requireApproval);
// Loan routes
router.get('/', getAllLoans);
router.get('/user/:userId', getUserLoans);
router.get('/:id', getLoan);
router.post('/', validateLoanCreation, createLoan);
router.put('/:id', authorize('ADMIN'), updateLoan);
router.delete('/:id', authorize('ADMIN'), deleteLoan);
router.put('/:id/status', authorize('ADMIN'), updateLoanStatus);
// Admin utility routes
router.put('/check-overdue', authorize('ADMIN'), checkOverdueLoans);
// All routes are protected
router.use(protect);
router.use(authorize('ADMIN'));
// Routes
router.get('/dashboard', getDashboardData);
router.get('/customers', getAllCustomers);
router.get('/loans', getAllLoansWithFilters);
```

```
router.get('/settings', getSystemSettings);
router.put('/settings', updateSystemSettings);
router.get('/renewals', getAllRenewalRequests);
router.put('/renewals/:id/status', updateRenewalStatus);
router.get('/statistics', getSystemStatistics);
module.exports = router;

// Notification routes
router.get('/user/:userId', getUserNotifications);
router.get('/user/:userId/unread-count', getUnreadCount);
router.put('/:id/read', markAsRead);
router.put('/user/:userId/read-all', markAllAsRead);
router.delete('/:id', deleteNotification);
// Admin routes
router.post('/', authorize('ADMIN'), createNotification);
module.exports = router;
```

renewals.js

```
const express = require('express');
const {
getAllRenewalRequests,
getUserRenewalRequests,
createRenewalRequest,
getRenewalRequest,
updateRenewalStatus
} = require('../controllers/renewalController');
const { protect, authorize, requireApproval } = require('../middleware/auth');
const { validateRenewalRequest } = require('../middleware/validation');
const router = express.Router();
// All routes are protected and require approval
router.use(protect);
router.use(requireApproval);
// Renewal routes
router.get('/', getAllRenewalRequests);
router.get('/user/:userId', getUserRenewalRequests);
```

```
router.get('/:id', getRenewalRequest);
router.post('/', validateRenewalRequest, createRenewalRequest);
router.put('/:id/status', authorize('ADMIN'), updateRenewalStatus);
module.exports = router;
```

transactions.js

```
const express = require('express');
const {
getAllTransactions,
getLoanTransactions,
getUserTransactions,
createTransaction,
updateTransactionStatus,
getTransaction
} = require('../controllers/transactionController');
const { protect, authorize, requireApproval } = require('../middleware/auth');
const { validateTransactionCreation } = require('../middleware/validation');
const router = express.Router();
// All routes are protected and require approval
router.use(protect);
router.use(requireApproval);
// Transaction routes
router.get('/', getAllTransactions);
router.get('/loan/:loanId', getLoanTransactions);
router.get('/user/:userId', getUserTransactions);
router.get('/:id', getTransaction);
router.post('/', validateTransactionCreation, createTransaction);
router.put('/:id/status', authorize('ADMIN'), updateTransactionStatus);
module.exports = router;
```

users.js

```
const express = require('express');
const {
getAllUsers,
```

```
getUser,
updateProfile,
deleteUser,
getUserStats
} = require('../controllers/userController');
const { protect, authorize } = require('../middleware/auth');
const router = express.Router();
// All routes are protected
router.use(protect);
// Admin routes
router.get('/', authorize('ADMIN'), getAllUsers);
router.get('/:id', getUser);
router.put('/:id', updateProfile);
router.delete('/:id', authorize('ADMIN'), deleteUser);
router.get('/:id/stats', getUserStats);
module.exports = router;
```

AdminDashboard.tsx

```
import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from
"@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter } from
"@/components/ui/dialog";
import {
Users,
TrendingUp,
AlertCircle,
DollarSign,
UserCheck,
FileText,
```

```
Settings,
LogOut,
Bell,
Search,
Filter,
MoreVertical,
CheckCircle,
XCircle,
User,
Edit,
Trash2,
Calendar,
Weight,
Gem,
RefreshCw
} from "lucide-react";
import { useToast } from "@/hooks/use-toast";
import { calculateAllValues, formatCurrency, validateGoldLoanData } from
"@/lib/goldCalculations";
import authService from "@/lib/authService";
import loanService from "@/lib/loanService";
import { authAPI, adminAPI } from "@/lib/api";
import { Loan, User as UserType, Transaction } from "@/lib/api";
const AdminDashboard = () => {
const navigate = useNavigate();
const { toast } = useToast();
const [selectedTab, setSelectedTab] = useState("overview");
const [loading, setLoading] = useState(true);
const [currentUser, setCurrentUser] = useState<UserType | null>(null);
const [dashboardData, setDashboardData] = useState<any>(null);
const [pendingUsers, setPendingUsers] = useState<UserType[]>([]);
const [allUsers, setAllUsers] = useState<UserType[]>([]);
const [allLoans, setAllLoans] = useState<Loan[]>([]);
const [recentTransactions, setRecentTransactions] = useState<Transaction[]>([]);
const [isCreateLoanOpen, setIsCreateLoanOpen] = useState(false);
const [isUserApprovalOpen, setIsUserApprovalOpen] = useState(false);
```

```
const [selectedUser, setSelectedUser] = useState<UserType | null>(null);
const [searchTerm, setSearchTerm] = useState("");
const [filterStatus, setFilterStatus] = useState("all");
const [newLoanData, setNewLoanData] = useState({
customer: "",
principalAmount: "",
interestRate: "12",
termDays: "30",
goldWeight: "",
goldPurity: "22",
goldRate: "6500",
collateral: "",
comments: ""
});
useEffect(() => {
const loadDashboardData = async () => {
setLoading(true);
// Check if user is authenticated and is admin
if (!authService.isAuthenticated()) {
navigate("/login");
return;
}
module.exports = router;
```

Notification.js:

```
const mongoose = require('mongoose');
const notificationSchema = new mongoose.Schema({
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: [true, 'User ID is required']
},
title: {
type: String,
required: [true, 'Notification title is required'],
```

```
trim: true,
maxlength: [100, 'Title cannot exceed 100 characters']
},
message: {
type: String,
required: [true, 'Notification message is required'],
trim: true,
maxlength: [500, 'Message cannot exceed 500 characters']
},
type: {
type: String,
enum: ['INFO', 'SUCCESS', 'WARNING', 'ERROR', 'LOAN_UPDATE', 'PAYMENT_DUE',
'APPROVAL'],
required: [true, 'Notification type is required']
},
isRead: {
type: Boolean,
default: false
},
data: {
type: mongoose.Schema.Types.Mixed,
default: {}
},
readAt: {
type: Date
},
expiresAt: {
type: Date,
default: function() {
// Notifications expire after 30 days
return new Date(Date.now() + 30 * 24 * 60 * 60 * 1000);
}
}
}, {
timestamps: true
});
```

```
notificationSchema.methods.markAsRead = function() {
this.isRead = true;
this.readAt = new Date();
return this.save();
};
notificationSchema.index({ userId: 1, isRead: 1, createdAt: -1 });
notificationSchema.index({ type: 1 });
notificationSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });
module.exports = mongoose.model('Notification', notificationSchema);
```

RenewalRequest.js:

```
const mongoose = require('mongoose');
const renewalRequestSchema = new mongoose.Schema({
loanId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Loan',
required: [true, 'Loan ID is required']
},
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: [true, 'User ID is required']
},
requestType: {
type: String,
enum: ['EXTENSION', 'RENEWAL', 'PARTIAL_PAYMENT'],
required: [true, 'Request type is required']
},

requestedExtensionDays: {
type: Number,
required: [true, 'Requested extension days is required'],
min: [1, 'Minimum extension is 1 day'],
max: [90, 'Maximum extension is 90 days']
},
```

```
reason: {
type: String,
required: [true, 'Reason for renewal is required'],
trim: true,
maxlength: [500, 'Reason cannot exceed 500 characters']
},
currentStatus: {
type: String,
required: true
},
originalAmount: {
type: Number,
required: true
},
originalDueDate: {
type: Date,
required: true
},
interestAccrued: {
type: Number,
default: 0
},
additionalInterest: {
type: Number,
default: 0
},
processingFee: {
type: Number,
default: 0
},
totalPayable: {
type: Number,
required: true
},
status: {
type: String,
```

```
enum: ['PENDING', 'APPROVED', 'REJECTED'],
default: 'PENDING'
},
processedAt: {
type: Date
},
processedBy: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User'
},
comments: {
type: String,
trim: true,
maxlength: [500, 'Comments cannot exceed 500 characters']
},
rejectionReason: {
type: String,
trim: true,
maxlength: [500, 'Rejection reason cannot exceed 500 characters']
}
}, {
timestamps: true
})
renewalRequestSchema.pre('save', function(next) {
if (this.isNew) {
this.totalPayable = this.originalAmount + this.interestAccrued + this.additionalInterest +
this.processingFee;
}
next();
});
renewalRequestSchema.index({ userId: 1, status: 1 });
renewalRequestSchema.index({ loanId: 1 });
renewalRequestSchema.index({ status: 1, createdAt: -1 });
module.exports = mongoose.model('RenewalRequest', renewalRequestSchema);
```

SystemSetting.js:

```
const mongoose = require('mongoose');
const systemSettingsSchema = new mongoose.Schema({
defaultInterestRate: {
type: Number,
required: true,
min: [1, 'Minimum interest rate is 1%'],
max: [50, 'Maximum interest rate is 50%'],
default: 12
},
currentGoldRate: {
type: Number,
required: true,
min: [1000, 'Minimum gold rate is ₹1,000 per gram'],
default: 6500
},
maxLoanAmount: {
type: Number,
required: true,
min: [1000, 'Minimum loan amount is ₹1,000'],
default: 1000000
},
minLoanAmount: {
type: Number,
required: true,
min: [1000, 'Minimum loan amount is ₹1,000'],
default: 10000
},
defaultLoanDuration: {
type: Number,
required: true,
min: [1, 'Minimum loan duration is 1 day'],
max: [365, 'Maximum loan duration is 365 days'],
default: 30
},
autoApprovalLimit: {
type: Number,
```

```
required: true,
min: [0, 'Auto approval limit cannot be negative'],
default: 50000
},
companyName: {
type: String,
required: true,
trim: true,
maxlength: [100, 'Company name cannot exceed 100 characters'],
default: 'GoldFlow Pro'
},
companyEmail: {
type: String,
required: true,
trim: true,
match: [/^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,3})+$/, 'Please enter a valid email'],
default: 'admin@goldflow.com'
},
companyPhone: {
type: String,
required: true,
trim: true,
match: [/^[6-9]\d{9}$/, 'Please enter a valid 10-digit phone number'],
default: '9876543210'
},
businessHours: {
type: String,
required: true,
trim: true,
default: '9:00 AM - 6:00 PM (Mon-Sat)'
},
emailNotifications: {
type: Boolean,
default: true
},
smsNotifications: {
```

```
type: Boolean,
default: true
},
overdueReminders: {
type: Boolean,
default: true
},
approvalAlerts: {
type: Boolean,
default: true
},
penaltyRate: {
type: Number,
min: [0, 'Penalty rate cannot be negative'],
max: [10, 'Maximum penalty rate is 10%'],
default: 2
},
processingFeeRate: {
type: Number,
min: [0, 'Processing fee rate cannot be negative'],
max: [5, 'Maximum processing fee rate is 5%'],
default: 1
},
renewalFeeRate: {
type: Number,
min: [0, 'Renewal fee rate cannot be negative'],
max: [3, 'Maximum renewal fee rate is 3%'],
default: 0.5
}
}, {
timestamps: true
});

// Ensure only one settings document exists
systemSettingsSchema.statics.getSettings = async function() {
let settings = await this.findOne();
```

```
if (!settings) {
settings = await this.create({});
}
```

Loan.js:

```
const mongoose = require('mongoose');
const loanSchema = new mongoose.Schema({
loanNumber: {
type: String,
required: true,
unique: true,
uppercase: true
},
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: [true, 'User ID is required']
},
customer: {
type: String,
required: [true, 'Customer name is required'],
trim: true,
maxlength: [100, 'Customer name cannot exceed 100 characters']
},
principalAmount: {
type: Number,
required: [true, 'Principal amount is required'],
min: [1000, 'Minimum loan amount is ₹1,000'],
max: [10000000, 'Maximum loan amount is ₹1,00,00,000']
},
interestRate: {
type: Number,
required: [true, 'Interest rate is required'],
min: [1, 'Minimum interest rate is 1%'],
max: [50, 'Maximum interest rate is 50%']
},
```

```
termDays: {
type: Number,
required: [true, 'Loan term is required'],
min: [1, 'Minimum loan term is 1 day'],
max: [365, 'Maximum loan term is 365 days']
},
startDate: {
type: Date,
required: [true, 'Start date is required'],
default: Date.now
},
dueDate: {
type: Date,
required: [true, 'Due date is required']
},
status: {
type: String,
enum: ['PENDING', 'ACTIVE', 'OVERDUE', 'COMPLETED', 'CANCELLED'],
default: 'PENDING'
},
collateral: {
type: String,
trim: true,
maxlength: [500, 'Collateral description cannot exceed 500 characters']
},
goldWeight: {
type: Number,
min: [0.1, 'Minimum gold weight is 0.1 grams']
},

goldPurity: {
type: Number,
min: [1, 'Minimum gold purity is 1%'],
max: [100, 'Maximum gold purity is 100%']
},
goldRate: {
```

```
type: Number,
min: [1000, 'Minimum gold rate is ₹1,000 per gram']
},
goldValue: {
type: Number,
min: [0, 'Gold value cannot be negative']
},
interestAmount: {
type: Number,
min: [0, 'Interest amount cannot be negative']
},
totalRepayment: {
type: Number,
min: [0, 'Total repayment cannot be negative']
},
comments: {
type: String,
trim: true,
maxlength: [1000, 'Comments cannot exceed 1000 characters']
},
createdBy: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: true
},

approvedBy: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User'
},
approvedAt: {
type: Date
}
}, {
timestamps: true
});
```

```
loanSchema.pre('save', function(next) {
if (this.isNew && !this.loanNumber) {
const timestamp = Date.now().toString().slice(-6);
const random = Math.floor(Math.random() * 1000).toString().padStart(3, '0');
this.loanNumber = `GL${timestamp}${random}`;
}
next();
});
loanSchema.pre('save', function(next) {
if (this.isNew && this.startDate && this.termDays) {
const dueDate = new Date(this.startDate);
dueDate.setDate(dueDate.getDate() + this.termDays);
this.dueDate = dueDate;
}
next();
});
loanSchema.pre('save', function(next) {
if (this.goldWeight && this.goldPurity && this.goldRate) {
this.goldValue = (this.goldWeight * this.goldPurity / 100) * this.goldRate;
}
if (this.principalAmount && this.interestRate && this.termDays) {
this.interestAmount = (this.principalAmount * this.interestRate / 100) * (this.termDays / 365);
this.totalRepayment = this.principalAmount + this.interestAmount;
}
next();
});
loanSchema.index({ userId: 1, status: 1 });
loanSchema.index({ loanNumber: 1 });
loanSchema.index({ dueDate: 1 });
loanSchema.index({ status: 1 });
module.exports = mongoose.model('Loan', loanSchema);
```

Notification.js:

```
const mongoose = require('mongoose');
```

```
const notificationSchema = new mongoose.Schema({
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: [true, 'User ID is required']
},
title: {
type: String,
required: [true, 'Notification title is required'],
trim: true,
maxlength: [100, 'Title cannot exceed 100 characters']
},
message: {
type: String,
required: [true, 'Notification message is required'],
trim: true,
maxlength: [500, 'Message cannot exceed 500 characters']
},
type: {
type: String,
enum: ['INFO', 'SUCCESS', 'WARNING', 'ERROR', 'LOAN_UPDATE', 'PAYMENT_DUE',
'APPROVAL'],
required: [true, 'Notification type is required']
},
isRead: {
type: Boolean,
default: false
},
data: {
type: mongoose.Schema.Types.Mixed,
default: {}
},
readAt: {
type: Date
},
expiresAt: {
```

```
type: Date,
default: function() {
// Notifications expire after 30 days
return new Date(Date.now() + 30 * 24 * 60 * 60 * 1000);
}
}
}, {
timestamps: true
});
notificationSchema.methods.markAsRead = function() {
this.isRead = true;
this.readAt = new Date();
return this.save();
};
notificationSchema.index({ userId: 1, isRead: 1, createdAt: -1 });
notificationSchema.index({ type: 1 });
notificationSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });
module.exports = mongoose.model('Notification', notificationSchema);
```

RenewalRequest.js:

```
const mongoose = require('mongoose');
const renewalRequestSchema = new mongoose.Schema({
loanId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'Loan',
required: [true, 'Loan ID is required']
},
userId: {
type: mongoose.Schema.Types.ObjectId,
ref: 'User',
required: [true, 'User ID is required']
},
requestType: {
type: String,
enum: ['EXTENSION', 'RENEWAL', 'PARTIAL_PAYMENT'],
```

```
required: [true, 'Request type is required']
},
requestedExtensionDays: {
type: Number,
required: [true, 'Requested extension days is required'],
min: [1, 'Minimum extension is 1 day'],
max: [90, 'Maximum extension is 90 days']
},
reason: {
type: String,
required: [true, 'Reason for renewal is required'],
trim: true,
maxlength: [500, 'Reason cannot exceed 500 characters']
},
currentStatus: {
type: String,
required: true
},
originalAmount: {
type: Number,
required: true
},
originalDueDate: {
type: Date,
required: true
},
interestAccrued: {
type: Number,
default: 0
},
```

APPENDIX 2

SNAPSHOTS

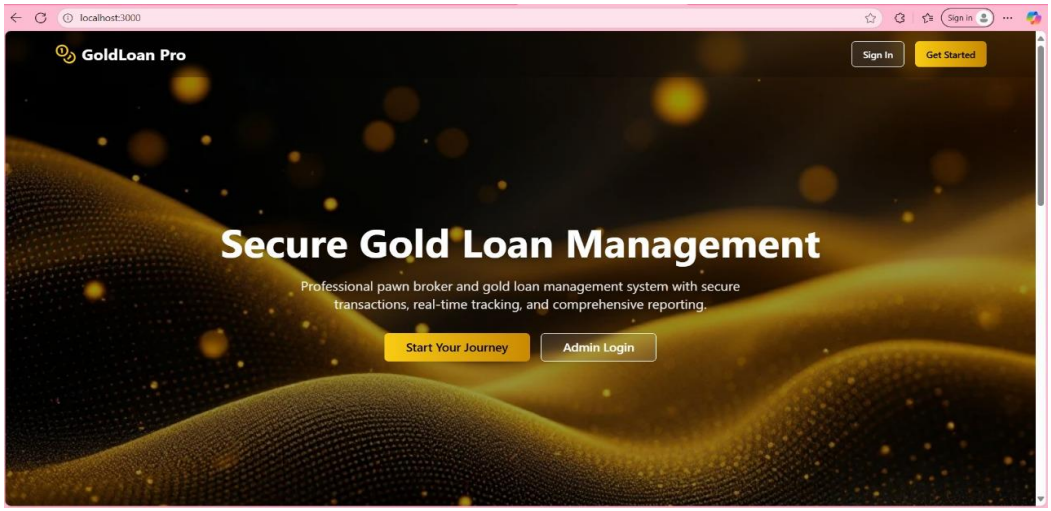

Figure A2.1 Home Page

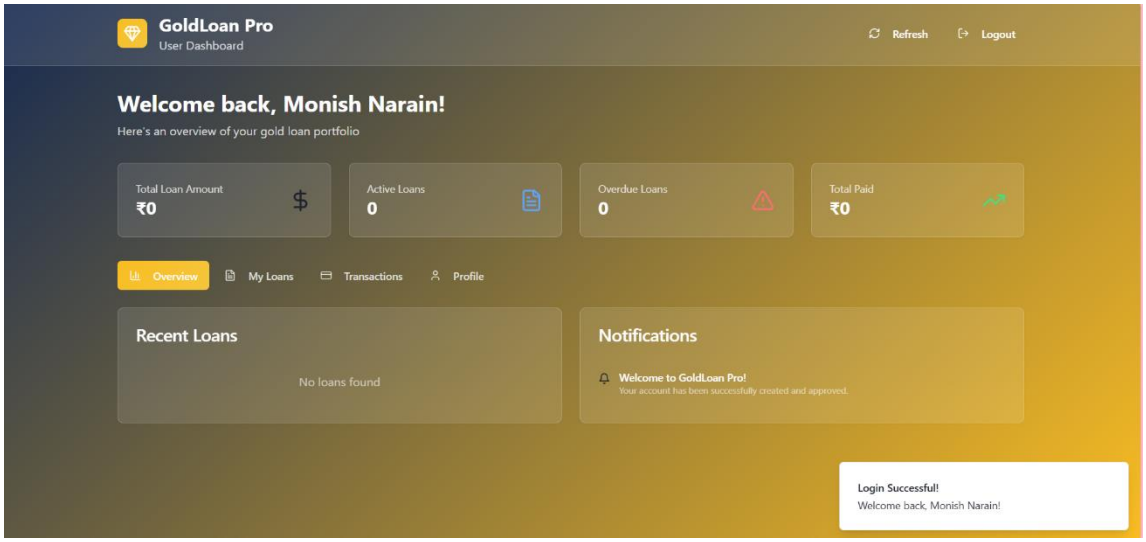

Figure A2.2 User Dashboard

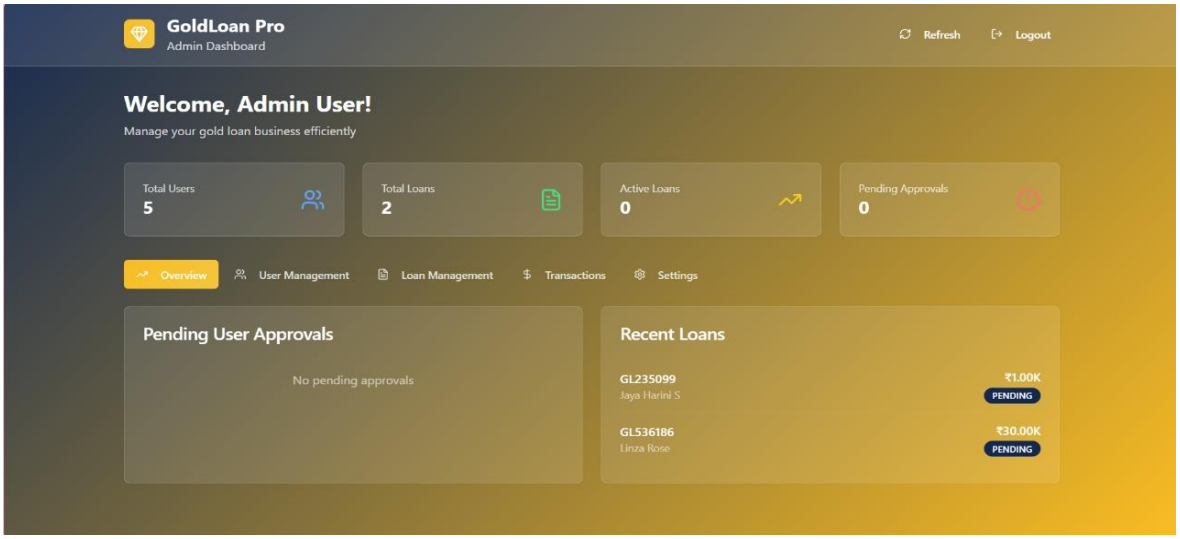

Figure A2.3 Admin Dashboard

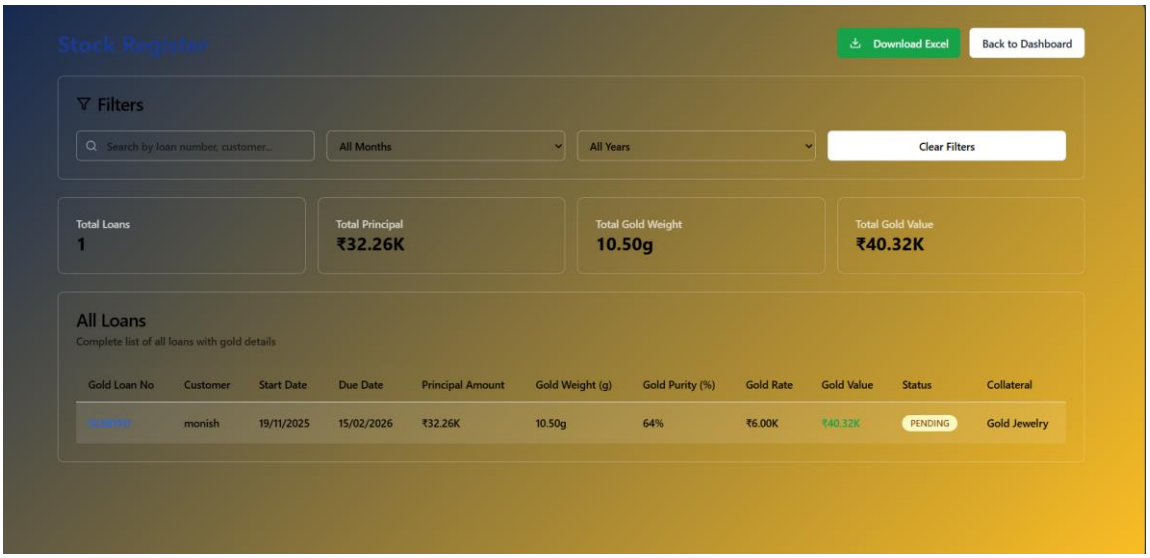

Figure A2.4 Stock Register

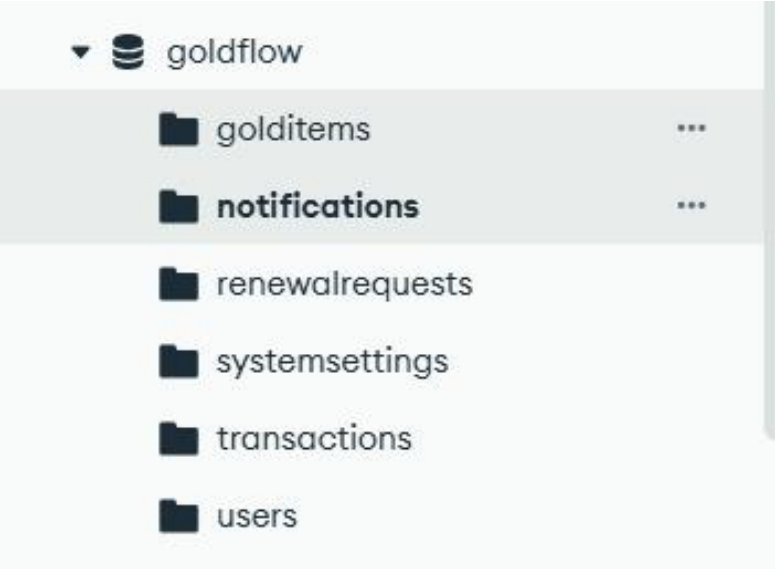

Figure A2.5 MongoDb

REFERENCES

1. Kamruzzaman, M. M. "Design and Implementation of Loan Management System using ISI Server, PHP, and MySQL." International Journal of Recent Technology and Engineering (IJRTE), vol. 8, no. 4, Nov. 2019, pp. 8017–8022.
2. Laguda, Ailen S. "Loans and Savings Management System for Bentuco National High School Faculty Cooperative." International Journal of Innovative Science and Research Technology (IJISRT), vol. 8, no. 5, May 2023, pp. 1076–1083.
3. Rey, W. P. "A Smart Contract-integrated Web-Based Loan Management System." Proceedings of the ACM International Conference on Information Systems and Software Engineering (ISSE), 2024, pp. 1–8.
4. Mazer, J., et al. "Design and Development of a Cloud-Based Loan Management System." International Journal of Advanced Engineering and Management (IJAEM), vol. 7, no. 9, Sept. 2025, pp. 358–377.
5. "MERN Stack Finance Tracker with Real-Time Analytics." International Journal of Scientific Research and Engineering Development (IJSRED), vol. 8, no. 3, May 2025, pp. 423–429.
6. Sharma, R., & Gupta, S. "A Secure Web-Based Banking and Loan Processing System Using Node.js and MongoDB." International Journal of Computer Applications (IJCA), vol. 181, no. 45, Feb. 2024, pp. 12–19.
7. Patel, D., and Singh, A. "Automation of Gold Loan Processing Using Modern Web Technologies." International Journal of Engineering Research & Technology (IJERT), vol. 10, no. 6, June 2023, pp. 554–560.
8. Ramesh, K., et al. "Cloud Deployment Strategies for Financial Applications Using MongoDB Atlas." International Journal of Cloud Computing and Services Science, vol. 12, no. 1, Jan. 2024, pp. 88–97.
9. Thomas, E., & Varghese, M. "Improving Loan Eligibility Prediction Using Machine Learning Approaches." International Journal of Computer Science and Mobile Computing (IJCSMC), vol. 11, no. 2, Feb. 2025, pp. 71–82.
10. Banerjee, S. "Digital Transformation of Gold Loan Services Through Online Platforms." Journal of Banking & Financial Technology, vol. 4, no. 3, Aug. 2024, pp. 221–230.